

Welcome to EECS 376 Discussion!

Jett Rosen - Sections 13 and 14

—

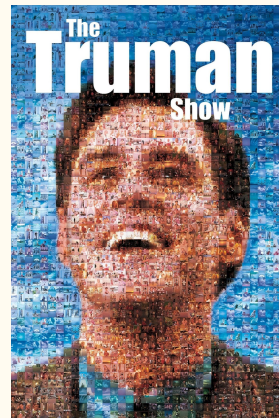
How it feels to draw a
square at the end of a proof



All feelings are temporary, you are loved,
you are enough, and you matter.

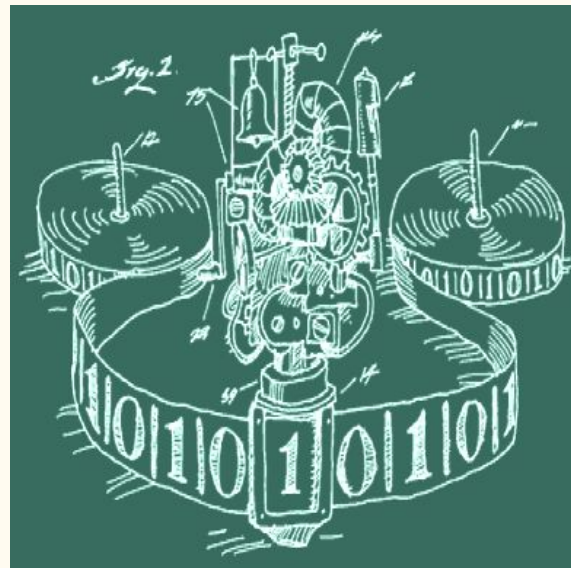
About Me

- I'm Jett Rosen
- 2nd semester SUGS CS student
- Second time GSI for 376 (+3 semester IA)
- Office Hours:
 - Subject to change, but currently:
 - Tuesday: 12:30-3pm LCSIB 2421
 - Thursday: 12:30-3:30pm LCSIB 2421
- Talk to me about: Machine learning, research, movies, hiking, art, Knicks basketball, One Piece, my dogs



So, why are we here?

- **Algorithms: How can we efficiently solve computational problems?**
 - **AKA: How to get a job**
- **Computability: Is a problem actually solvable?**
 - **AKA: How to tell your boss you can't do something**
- **Complexity: How quickly can we solve a problem?**
 - **AKA: How to explain to your boss why your code is slow**
- **Randomness: How can we use randomness to solve computational problems?**
 - **AKA: How to (probably) make your boss happy**
- **Cryptography: How can we share secret information over public channels?**
 - **AKA: How to hide your conversations from your boss**



Student Testimony

I also felt like sharing that I was able to use my knowledge of the birthday problem, scheduling problem, and NP-hard problems at my internship to work faster and make my algorithms more efficient! I was really surprised that a theory class would actually aid my thinking and implementation in real world software engineering.

Advice From a 6x EECS 376 Veteran

- Start the homework early
 - I *cannot* stress this enough
- Find a routine that works for you
 - I would start HW on Friday, work throughout the weekend, and then go to OH Monday and Tuesday for the problems I struggled with
- Please take advantage of every resource in this class:
 - Piazza, office hours, discussions, lecture notes, discussion worksheets and solutions
 - If you feel lost, come to office hours and talk to us! A lot of this material will not make sense the first time you see it, but it's rewarding to understand so let us help you!
- We are here to help you learn!

Discussion Format

- I will take attendance, talk at you for a little bit
- Split into small groups (2-3 ideal, 4 max) to work on specified questions
 - Find a group for yourself in the first few weeks!
 - If you don't, I'll assign one around discussion 3 or 4
- I'll be around for any questions that come up while working, and then we'll go over the problem as a group
- Slides will be posted in 376 Google Drive/Discussions/Slides/Jett
 - If not, call me out on Piazza

Logistics

- Attendance QR code will be on a later slide, make sure you fill it out!
- **Completely optional** exit ticket at the end of discussion
 - But fill it out :)
- If you attend discussion 10 of the 14 weeks, you will receive full credit for discussion attendance (3% of grade).
 - Otherwise, your combined exam score will count as your score for this component of the final grade.
 - So go to discussion
- Homework 1 due Wednesday
 - Start early!

Public Service Announcement

- Do not use ChatGPT (or any LLM) to do schoolwork
 - Doing so quite literally causes your brain to atrophy
- You will **not** enjoy EECS 376 if you do not do the work yourself
- In this class, you will learn the most while completing the homeworks
 - If you can do the homework problems, you can do the test problems!

First Day Icebreaker!

- Group up with ~ 2 people around you, introduce yourselves!
- Discuss a “hot take” you swear by
- In a few minutes, I’ll ask everyone to share



Quick Review: Asymptotic Notation

- **Definition:** We say that $f(n) = O(g(n))$ if there exist positive constants c , $n_0 > 0$ such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$

- **Definition:** We say that $f(n) = \Omega(g(n))$ if there exist positive constants c , $n_0 > 0$ such that:

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0.$$

- **Definition:** We say that $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Quick Review: Proof Methods

- **Direct proof:** To prove $P \Rightarrow Q$, we assume that P is true, then write a sequence of logical consequences directly showing that Q must also be true.
- **Proof by contrapositive:** Another way of proving $P \Rightarrow Q$ is to (directly) prove the logically equivalent contrapositive of the statement, which is $\neg Q \Rightarrow \neg P$.
- **Proof by contradiction:** To prove P is true, we first assume that P is false, then derive some logical contradiction.

Quick Review: Induction

- Used to prove a statement over a countably infinite set
- Three main components:
 - **Base Case:** Prove that a statement is true in the most basic case, for natural numbers, this tends to be 0 or 1.
 - **Induction Hypothesis:** Assume the statement is true for an arbitrary element (number), k
 - **Inductive Step:** Assuming your induction hypothesis, prove that the statement is true for $k+1$ (or the next element in your set)
- Strong Induction: typical induction proves $P(k) \rightarrow P(k+1)$, strong induction proves $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$.

The Potential Method

- The potential method is a useful technique to reason about the amount of time an algorithm takes before halting
- **Definition:** A function $s: A \rightarrow \mathbb{R}$ that maps the "state" of the algorithm (i.e., its internal variables) to a **(finite) real number** is a valid potential function if:
 - s **strictly decreases** by at least **some fixed positive constant** $C > 0$ with **every "unit of work"**
 - ensures that $s \leq B$ will eventually occur within a **finite** number of work units
 - s has some **finite lower bound** B , i.e., the algorithm is guaranteed to terminate once $s \leq B$
 - ensures that the algorithm **will terminate** within the **finite** number of works units

Potentially Some Tips and Tricks

- You define the "unit of work"
 - If we are trying to prove a function that uses a loop will eventually terminate, one iteration usually works, **but not always**
- Ensure **both conditions** are satisfied when defining a potential function
- Before writing a potential function, I would try to get some sense of what the algorithm is doing, so that the termination condition makes more sense
 - Oftentimes functions have a natural notion of potential

Exit Ticket

